

Graph Laplacians and Least Squares on Graphs

Anil N. Hirani
Mathematics

Univ. of Illinois at Urbana-Champaign
Email: hirani@illinois.edu

Kaushik Kalyanaraman
Computer Science

Univ. of Illinois at Urbana-Champaign
Email: kalyana1@illinois.edu

Seth Watts
Mechanical Science and Engineering

Univ. of Illinois at Urbana-Champaign
Email: watts2@illinois.edu

Abstract—There are several classes of operators on graphs to consider in deciding on a collection of building blocks for graph algorithms. One class involves traditional graph operations such as breadth first or depth first search, finding connected components, spanning trees, cliques and other subgraphs, operations for editing graphs and so on. Another class consists of linear algebra operators where the matrices somehow depend on a graph. It is the latter class of operators that this paper addresses. We describe a least squares formulation on graphs that arises naturally in problems of ranking, distributed clock synchronization, social choice, arbitrage detection, and many other applications. The resulting linear systems are analogous to Poisson's equations. We show experimental evidence that some iterative methods that work very well for continuous domains do not perform well on graphs whereas some such methods continue to work well. By studying graph problems that are analogous to discretizations of partial differential equations (PDEs) one can hope to isolate the specific computational obstacles that graph algorithms present due to absence of spatial locality. In contrast, such locality is inherent in PDE problems on continuous domains. There is also evidence that PDE based methods may suggest improvements suitable for implementation on graphs.

I. INTRODUCTION

Scientific computing problems almost always eventually require the solution of linear algebra problems at some level. Due to this pervasiveness of linear algebra, software packages that implement building blocks for numerical linear algebra have been very successful in making this aspect of scientific computing efficient, portable and easier to work with [1]–[4]. There are several recent and ongoing efforts for defining an analogous collection of building blocks for graph algorithms [5], [6].

Many important scientific and engineering applications were and still are modeled using partial differential equations (PDEs) on domains that are subsets of $\mathbb{R}^2$ or $\mathbb{R}^3$. (For convenience we will refer to these as geometric domains.) The spatial locality inherent in differential operators on such domains induces a locality and structure in the matrices representing the discretized differential operators. This has been exploited widely in implementation of solvers for problems arising from PDEs.

Graph algorithms and operators can be categorized into several classes. One class consists of traditional graph algorithms such as breadth first or depth first search, algorithms

for finding connected components, spanning trees, cliques, or other types of subgraphs, editing operations on graphs, and so on. These operators and algorithms are one of the oldest parts of computer science and many algorithms for graphs have been standardized in terms of operations on graphs [7]. Thus it makes sense to look for which building blocks can be implemented in a portable and efficient manner for computational purposes.

Another class of graph operators consist of linear algebra operators whose matrix representations arise from graphs in some way. This paper addresses a class of least squares problems belonging to this category. The problems we consider lead to systems of equations that are analogous to Poisson's equation but are posed on graphs. (The analogy will be made precise later in the paper.) The application areas where these least squares problems on graphs arise include ranking, social choice and arbitrage [8], distributed synchronization of clocks [9], [10], and many others. We show experimental evidence that some iterative numerical linear methods that work well in the case of the analogous traditional PDE problems on geometric domains fail to perform well on the corresponding graph least squares problems. Some other methods perform well in both situations.

In considering such problems on graphs we argue for the following point of view. There is a lot of collected expertise on numerical linear algebra in scientific computing and numerical analysis arising from discretizations of PDEs. By posing problems on graphs that are analogous to PDE problems one may be able to understand the special computational difficulties that graphs introduce. This is in keeping with good engineering practice of “change one thing at a time” when trying to understand behavior of a system with many inter-related parts. Moreover, there is evidence that techniques and ways of thinking developed in PDE-related scientific computing are relevant for improving performance of graph algorithms. One such example is the construction of hierarchical solvers for mixed-form graph Laplacian problems (analogous to finite element exterior calculus [11]) that attempts to achieve optimal efficiency [12].

A. Ranking and pairwise comparisons

In this paper we will motivate the least squares problems on graphs using ranking based on pairwise comparisons as

a prototype application. (This setting is different from that of the ranking that Google’s PageRank algorithm typically computes [13] and is similar to that of ranking of pairwise data based on random walks [14].) One should bear in mind that the same problem structure emerges in a wide array of applications alluded to earlier. In the particular ranking problem considered we want to rank items, of which some pairs have been compared. The formulation we use (and which we did not invent) leads to a least squares computation on graphs, and a deeper analysis requires a second least squares solution [8]. The topology of the graph plays a role, in a way that will be made precise later. The usual graph Laplacian plays a central role in the first least squares problem. The key actor in the second problem is a combinatorial analog of another Laplacian well-studied in analysis.

The formulation as two least squares problems is akin to finding the gradient part of a vector field and its curl part. (This is the Helmholtz decomposition in vector calculus.) That in turn, is related to solving an elliptic partial differential equation. The setting for the ranking problem however, is obviously different from that for vector fields and differential equations since the domains are not geometric but are graphs.

B. Contributions and goals of this paper

We compare iterative Krylov methods with algebraic multigrid methods for a class of least squares problems on graphs. We show experimental evidence that algebraic multigrid is not competitive for these graph problems. The use of a PDE type problem brings a different point of view to the task of selecting building blocks for graph algorithms, and understanding their limitations and opportunities. It may also permit the use of techniques developed in the PDE-related literature to related problems on graphs [12].

A second motivation is to draw attention to least squares problems on graphs as a potential source for benchmarks for graph problems. The high performance computing community now maintains a Graph 500 list [15]. This is like the Top 500 list of supercomputers but focused on graph problems. Current benchmarks include three classes of problems, namely, search, shortest path, and edge-oriented problems. Efforts to include social network problems are underway.

In the field of high performance computing, problems like least squares and linear systems for elliptic partial differential equations have always been an important source of problems. These have led to many developments, such as in domain decomposition, preconditioners, and iterative methods. The problem of least squares on graphs also involves Laplacians, but these are graph Laplacians and other Laplacians on general graphs. When very large ranking problems on diverse architectures are attempted, it is likely that new developments will be needed. At the same time, the problems are easy to set up and some old codes from

differential equations can be used right away. Thus the least squares ranking on graphs is a good crossover problem and a bridge from Top 500 to Graph 500.

II. APPLICATIONS OF LEAST SQUARES ON GRAPHS

Least squares on graphs can model a diverse set of applications including ranking of sports teams [16], inconsistency analysis of ranking, social choice and arbitrage [8], distributed synchronization of clocks across a network [9], [10], and many others. For the purpose of exposition, in what follows we fix our application to be pairwise ranking on graphs.

We are given a set of items to be ranked and some real-valued pairwise comparisons wherein not all possible pairs have been compared. Each given pairwise comparison represents how much one alternative in a pair is preferred over the other. This data can be represented, for example, as a weighted directed graph, where each comparison between a pair of items is represented by two edges of opposite direction and weights between the items that are equal in magnitude but opposite in sign [8]. This leads to a skew-symmetric 2-tensor representation of comparisons. However, there is an equivalent simple, weighted, undirected graph whose edges are oriented. The edge orientations are arbitrary, and the pairwise score simply changes sign if the opposite edge orientation is used. Without loss of generality, we will usually only consider connected graphs. (Multiple component graphs result in independent ranking problems, one for each component.)

One version of the ranking problem is to find real-valued scores for the vertices, which implies their global rank order, such that the values represent the strength of the rank. The task translates to finding vertex values whose differences are the edge values. However, it is not always possible to compute this exactly. Every cycle in the graph has the potential to make existence of such vertex values impossible if the edge values, taken with signs, do not add up to zero as the cycle is traversed. In this case the closest possible global ranking is the vertex value assignment whose differences reproduce the pairwise edge data in the least squares sense. This is a very simple and an old idea that was used for ranking football teams [16]. The residual, i.e., the part of the edge data that could not be matched, represents inconsistencies in the pairwise data [8].

The edge data is called consistent if the sum of edge weights (taking orientations into account) around every cycle is zero. Consistency is equivalent to having a zero residual in the least squares sense. Least squares ranking can still be computed for inconsistent edge data. For example, the data illustrated in the leftmost graph in Fig. 1 is consistent while the other two are not. However, a least squares ranking is possible on each. In particular, *A* will be the winner and *C* the loser in the first two cases, and there will be a three-way tie in the last one.

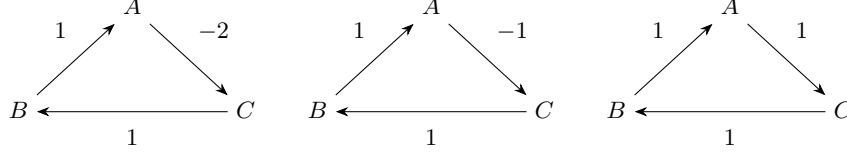


Figure 1: Examples of consistent (left) and inconsistent (middle and right) pairwise comparisons when evaluating three alternatives.

An extension of [16] was given in [8] in which the authors examined the residual, decomposing it into local and global inconsistencies, using a second least squares formulation. This time a 3-tensor, i.e., a 3-dimensional matrix, is involved as an unknown in their formulation. As we will show, equivalent least squares problems can be formulated using vectors rather than matrices or 3-tensors to represent the data and unknowns.

Another innovation of [8] was to pose the ranking problem as a discrete *Hodge decomposition* of the pairwise data treated as a cochain on a simplicial 2-complex (terminology defined in Section III). Using the same point of view will help us clarify the connection to vector field decomposition, elliptic partial differential equations, and topology of complexes.

We emphasize that the idea in [16] is the use of least squares to find values for each vertex in the graph, given (generally inconsistent) edge data. This is very different from the use of least squares to fit data to a model equation in the sense of statistical regression. It is better to think of the methods of [16] and [8] as being orthogonal projections or direct-sum decompositions, which is the viewpoint that we will take in this paper.

III. PRELIMINARIES

We first provide a brief overview of some notions required for viewing the considered least squares problems on graphs as analogous to Poisson's equations. First, we recall the graph Laplacian and provide a description of generalized graph Laplacian matrices. These are, in turn, related to notions of scalar and vector Laplacians in vector calculus. We also provide a concise introduction to some language borrowed from algebraic topology and Hodge theory for describing combinatorial objects on graphs that help in the formulation of the least squares problems. Finally, we will exhibit the equivalence between these problems and the Hodge decomposition.

A. Graph Laplacians

The Laplacian L on a graph G is a well known operator defined by:

$$L(u, v) = \begin{cases} d(u) & u = v, \\ -1 & u, v \text{ are incident on each other,} \\ 0 & \text{otherwise,} \end{cases}$$

where u, v are any vertices of G , and $d(u)$ is the edge incidence degree of vertex u . L is related to both the vertex incidence and edge incidence matrices of the graph, A and S , respectively. A encodes the vertex adjacency information, i.e., $A(u, v) = 1$ if there is an edge joining vertices u and v in G while S provides the vertex-edge incidence, i.e., $S(u, e) = 1$ and $S(v, e) = -1$ for an edge e between vertices u and v oriented from u to v . Note that the orientation in case of non-directed graphs is an arbitrary choice. Note that this definition of graph Laplacian is different from one that appears in [17] which is sometimes referred to as the normalized graph Laplacian [18]. These various matrices are related to each other by:

$$L = D - A, \quad L = S S^T,$$

where D is the square diagonal matrix of vertex degrees, i.e., $D(u, u) = d(u)$. Also, note that in the setting of algebraic topology, the matrix S is the boundary operator ∂_1 mapping 1-chains or data on edges to 0-chains or data on vertices. This description allows for generalizing the vertex-edge incidence matrices to higher degree k -chains which usually have no easy analogs in the setting of graphs. For instance, ∂_2 is the operator mapping 2-chains to 1-chains which can be thought of as a k -clique-edge incidence matrix for a fixed $k \geq 3$. We shall henceforth not reference these incidence matrices using their graph notations but instead use ∂_1 and ∂_2 .

B. Simplicial complexes and Hodge decomposition

A collection of n distinct vertices is referred to as a n -simplex or *simplex*. Simplices are essentially a finite set of vertices. The *dimension* of a simplex is its cardinality (as a set). An *orientation* of the simplex is an equivalence class of ordering of its simplices. An *abstract simplicial complex* is a collection of simplices with the property that if a simplex belongs to the complex, then so does every non empty subset of that simplex. The dimension of a complex is the highest cardinality of its simplices. An *oriented complex* is one for which an orientation is specified for all its simplices. For our purposes, the choice of orientations is arbitrary. A standard reference for a detailed exposition of these notions is [19]. Thus, in this description, a graph is a 1-complex consisting of 0-simplices (vertices) and 1-simplices (edges). Adding in the cliques of the graph leads to enriching this 1-complex to

$$\text{functions} \xrightleftharpoons[\text{div}]{\text{grad}} \text{vector fields} \xrightleftharpoons[\text{curl}]{\text{rot}} \text{functions} \quad (1)$$

$$\text{functions} \xrightleftharpoons[\text{div}]{\text{grad}} \text{vector fields} \xrightleftharpoons[\text{curl}]{\text{curl}} \text{vector fields} \xrightleftharpoons[\text{grad}]{\text{div}} \text{functions} \quad (2)$$

a higher dimensional one. For instance, adding all 3-cliques (triangles) in the graph would lead to a *simplicial 2-complex*.

A Hodge decomposition, for our purposes, can be thought of as a generalization of the Helmholtz decomposition of vector fields to include the cases where the domain has nontrivial topology. This means that the decomposition of a vector field into a gradient part and a curl part additionally includes a *harmonic* part which is the kernel of the 1-Laplacian operator (see Section III-C). Furthermore, these three components of the vector field are mutually orthogonal to each other under the standard inner product on these vector spaces.

C. Vector calculus analogies

Laplacians on graphs are related to scalar and vector Laplacians in vector calculus; refer to (1) and (2) which are examples of what are referred to as de Rham complexes. The matrix ∂_1^T is a graph analog of the gradient operator and ∂_1 is the analog of negative divergence operator. Similarly, ∂_2 and ∂_2^T are analogues of either two dimensional vector curl and scalar curl operators, respectively or the more familiar three-dimensional curl operators. These matrices also have the property that $\partial_2^T \partial_1^T = 0$ similar to the vector calculus identities $\text{curl} \circ \text{grad} = 0$ and $\text{div} \circ \text{curl} = 0$.

The combinatorial graph Laplacian L is the discrete analog of the usual scalar Laplacian in vector calculus $\Delta_0 = -\text{div} \circ \text{grad}$. In vector calculus in $\mathbb{R}^2$, the vector Laplacian is the operator $\Delta_1 = \text{curl} \circ \text{curl} - \text{grad} \circ \text{div}$ as shown in (1). The combinatorial analog is $\partial_1^T \partial_1 + \partial_2 \partial_2^T$. If we did not include the triangles (or cells) and considered G only as a 1-dimensional complex, ∂_2 would be the zero matrix. Then the 1-Laplacian would be $\partial_1^T \partial_1$ which is sometimes called the edge Laplacian in graph theory. There is no name for Δ_2 in graph theory. But this 2-Laplacian $\partial_2^T \partial_2$ is the graph theoretic analog of the 2-Laplacian in Hodge theory [20] and finite element exterior calculus [11] on a 2-dimensional manifold.

Finally, the combinatorial analogue of functions and vector fields are objects known as *cochains*. A 0-cochain would be a discrete function defined on vertices, a 1-cochain would be related to vector fields and so on. The combinatorial version of (1) and (2) would be what is called as a cochain complex. However, for simplifying purposes, these cochains can be thought of as being the same as chains described in Section III-A. Indeed, there exist isomorphisms (as vector spaces) between chain complexes and cochain complexes

but in all further discussion, we will think of cochains as values associated with either the vertices, edges or cliques and make no distinction of them from chains.

D. Least squares and Hodge decomposition

We first consider the case when given pairwise data, i.e., a 1-cochain ω , has components along both row space $\text{im } \partial_1^T$ of ∂_1 and the column space $\text{im } \partial_2$ of ∂_2 . In this case, the Hodge decomposition, least squares, and normal equations are equivalent.

In a least squares problem $Ax \simeq b$, to minimize $\|b - Ax\|_2^2$ as a function of x , a necessary condition is that the gradient $A^T(b - Ax)$ be zero which yields the normal equations. Thus residual minimization implies the normal equations. For the converse, a sufficient condition is that the Hessian matrix $(2A^T A)$ be positive definite which is equivalent to requiring A be full rank [21, page 110]. In the case of graphs, however, the matrices ∂_1^T , ∂_2 will in general have nontrivial kernels. For example, the constant functions on the vertices constitute the null space $\ker \partial_1^T$ of ∂_1^T .

We shall now formulate the least squares problem more generally as follows. We will require for a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $x_* \in \mathbb{R}^n$, if $A^T A x_* = A^T b$ that $x_* \notin \ker A$. (Here m and n are appropriate dimensions, for example, in the first least squares problem, m will be the number of vertices and n the number of edges.) This would then be equivalent to x_* minimizing the residual norm $\|b - Ax\|_2$ for all $x \in \mathbb{R}^n$. This follows from the dot product $\langle b - Ax_*, Ax_* \rangle = \langle A^T(b - Ax_*), x_* \rangle = 0$. This means either $b - Ax_* = 0$, or that the vectors $b - Ax_*$ and Ax_* are orthogonal. The latter means that the shortest distance from b to $\text{im } A$ is achieved by Ax_* .

Given 1-cochains ω, h , 0-cochain α , and 2-cochain β , with $\alpha \notin \ker \partial_1^T$ and $\beta \notin \ker \partial_2$, the following are equivalent.

(i) Hodge Decomposition (HD):

$$\begin{aligned} \omega &= \partial_1^T \alpha + \partial_2 \beta + h, \\ h &\in \ker \Delta_1. \end{aligned} \quad (3)$$

(ii) Least Squares (LS): $a = \alpha, b = \beta$, and $s = h$ are optimal values of the two least squares problems:

$$\min_a \|r\|_2 \quad \text{such that} \quad r = \omega - \partial_1^T a, \quad (4)$$

$$\min_b \|s\|_2 \quad \text{such that} \quad s = r_* - \partial_2 b, \quad (5)$$

where r_* is the minimizing residual for (4). In least squares short hand notation one would write the two problems as $\partial_1^T a \simeq \omega$ and $\partial_2 b \simeq r_*$.

(iii) Normal Equations (NE): $a = \alpha$ and $b = \beta$ are a solution of the two linear systems:

$$\partial_1 \partial_1^T a = \partial_1 \omega, \quad (6)$$

$$\partial_2^T \partial_2 b = \partial_2^T r_*, \quad (7)$$

where r_* is the residual $\omega - \partial_1^T \alpha$.

The three terms in the Hodge decomposition in (3) are mutually orthogonal. This is easy to see. It follows simply from the fact that $\partial_1 \partial_2 = 0$ and from the definition of the harmonic part ($\ker \Delta_1$). For example, given an 1-cochain ω , if it has a nonzero harmonic part h , then $\langle h, \partial_1^T \alpha \rangle = \langle \partial_1 h, \alpha \rangle = 0$ since h is in $\ker \partial_1 \cap \ker \partial_2^T$.

Due to these orthogonality conditions, it is easy to see that the second least squares problem, which is $\partial_2 b \simeq r_*$, can also be written as $\partial_2 b \simeq \omega$. Similar changes can be made from r_* to ω in the second systems in all the formulations above. For ease of reference, below we write the least squares and normal equations using ω instead of r_* all in one place. The least squares systems are:

$$\partial_1^T a \simeq \omega, \quad (8)$$

$$\partial_2 b \simeq \omega, \quad (9)$$

and the corresponding normal equations:

$$\partial_1 \partial_1^T a = \partial_1 \omega, \quad (10)$$

$$\partial_2^T \partial_2 b = \partial_2^T \omega. \quad (11)$$

From the definition of the Laplacians in Section III-C, these can be written as $\Delta_0 a = \partial_1 \omega$ and $\Delta_2 b = \partial_2^T \omega$.

E. Interpretation in terms of ranking

Given any pairwise comparison data ω , by (3), there exists data on vertices α (the vertex potential or ranking), a 2-cochain β and a harmonic field $h \in \ker \Delta_1$ such that $\omega = \partial_1^T \alpha + \partial_2 \beta + h$. The α term is the scalar potential that gives the ranking. The β term is defined on cells and captures the local inconsistency in the data. The harmonic part contains the inconsistency that is present due to cycles longer than the maximum number of sides in the cells. If only 3-cliques (triangles) are considered as the 2-dimensional cells, then any inconsistency in cycles of length four or more will be captured in the harmonic part. The following example should make some of this more apparent.

Fig. 2 shows an example of solving the first least squares problem (4). It is just as easy to work with disconnected graphs, so we show a graph with two components. The values on the edges is the given data ω . The vertex potential values α are written in italics. Note that in the straight line part of the graph which does not involve a cycle, it is clear what the vertex potential should be (up to an additive constant). There will be no residual in this case. The first triangle after the straight line part is consistent because the value on the diagonal edge is the sum of the values on the other two sides which are oriented appropriately. The other

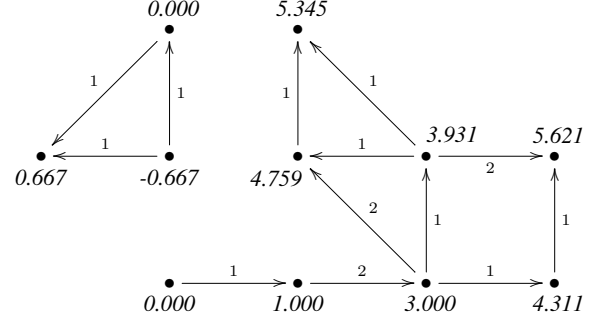


Figure 2: The results of the first least squares problem in (4) on an example graph of pairwise comparisons (the numbers on the edges). The graph has two connected components which are ranked independently. The solution of the least squares problem provides the ranking of vertices shown as vertex labels here.

triangles and the square cycle are all inconsistent. Here only triangles are chosen as the 2-dimensional cells, so the β part will be the inconsistency associated with the triangles if the second problem were also to be solved. The harmonic part h would be the inconsistency in the square cycle. Note that because of two connected components the dimension of $\ker \Delta_0 = \partial_1 \partial_1^T$ will be two. Fixing one vertex value in each of the two components and deleting the appropriate row and column will make the normal equations system (6) nonsingular.

IV. BUILDING BLOCKS FOR LEAST SQUARES ON GRAPHS

In most areas of numerical analysis, the computation that sits at the heart of the solution method is usually the solution of a linear system. Least squares ranking on graphs is no different. This section is about our numerical experiments for testing the accuracy and speed of linear system solvers for the first and second least squares problems of ranking on graphs.

Our main focus has been on two models of random graphs, the Erdős-Rényi model [22], one of the most studied random graph models, and the Barabási-Albert model [23] for power law graphs. The Erdős-Rényi model has two parameters, namely, the number of nodes and the probability that any pair of nodes is connected by an edge. We will refer to this probability as the *edge density*. However, this model does not account for the phenomena of clustering as seen in many networks in societies or for power law degree distributions as seen in internet graphs, reaction networks of molecules in a cell or airline flight networks. These are called scale-free networks or graphs and this feature is captured by the model of Barabási and Albert. The generative model is often implemented as a random process, although in practice these are typically not random graphs.

In the numerical analysis community there is a lot of accumulated experience on solving linear systems that arise

from partial differential equations. Studies of systems arising from graphs are less common but appearing with increasing frequency. For least squares ranking on general graphs there is no guidance available in the literature. Ours is an attempt to fill that gap. The fact that the underlying problem is coming from a random graph introduces some new challenges as will be demonstrated in our experiments.

We only consider iterative linear solvers here, though sparse direct solvers might be worth considering. We used an iterative Krylov method suitable for symmetric systems and one that is suitable for rectangular systems. We also used algebraic multigrid using smoothed aggregation and Lloyd aggregation. The results are discussed in Sections IV-B and IV-C. An especially attractive feature of all these methods is their ability to ignore the kernel of the operator involved. We do not know how direct methods could be made to do that for the second least squares problem which can have a large dimensional kernel depending on the graph topology. In direct solvers, for the first least squares problem the nontrivial kernel can be handled by fixing the value at a single vertex, just as is done by fixing pressure at a point in fluid problems.

A. Methodology

All numerical experiments were done using the Python programming language. The Krylov linear solvers used were those provided in the SciPy module [24]. The algebraic multigrid used was the one provided in PyAMG [25]. The simplicial complexes and boundary matrices were created using the PyDEC module [26]. The errors and times required by various solvers is generated as an average over multiple trials. This is done to minimize the influence of transient factors that can affect performance of a computer program. All experiments were carried out on a computer with a 2.53 GHz Intel Core 2 Duo processor and 4 GB of memory.

In each case, a graph G with the desired number of nodes N_0 and other desired characteristics (such as edge density in the case of Erdős-Rényi graphs) is first generated by a random process. We then find all the 3-cliques in the graph and create a simplicial complex data structure for the resulting 2-complex. Let the number of edges and triangles in G be N_1 and N_2 .

A random ranking problem instance is created for this complex. This entails creating a random 1-cochain representing the comparison data on edges. The point of these experiments is to compare the accuracy and efficiency of the tested methods, and so the Hodge decomposition of this 1-cochain has to be known in advance. In other words, a random problem instance is a 1-cochain ω such that there are random but known α on vertices, β on 2-cliques, and $h \in \ker \Delta_1$ with $\omega = \partial_1^T \alpha + \partial_2 \beta + h$.

To create the random gradient part $\partial_1^T \alpha$ and the random curl part $\partial_2 \beta$, we simply pick random vectors with N_0 entries for α and N_2 entries for β . To compute a random

harmonic part, we compute the Hodge decomposition of a random 1-cochain ρ by solving two least squares problems $\partial_1^T a \simeq \rho$ simultaneously.

$$\begin{bmatrix} \partial_1^T & \partial_2 \end{bmatrix} x \simeq \rho, \quad (12)$$

where the matrix $\begin{bmatrix} \partial_1^T & \partial_2 \end{bmatrix}$ is formed by horizontally stacking ∂_1^T and ∂_2 matrices. If x is the solution of this least squares problem, then the residual $\rho - \begin{bmatrix} \partial_1^T & \partial_2 \end{bmatrix} x$ is harmonic. This follows from the fact that the residual $b - Ax$ in a least squares problem $Ax \simeq b$ is orthogonal to the $\text{im } A$. Hence $\rho - \begin{bmatrix} \partial_1^T & \partial_2 \end{bmatrix} x \in \ker \begin{bmatrix} \partial_1^T & \partial_2 \end{bmatrix}^T = \ker \partial_1 \cap \ker \partial_2^T = \ker \Delta_1$, thus is a harmonic.

B. Iterative Krylov methods

We experimented with many iterative Krylov solvers but here report only results from using conjugate gradient (CG). As stated earlier, the Laplacian systems we consider on graphs are symmetric and positive semidefinite for which CG can be used to obtain a solution even in the presence of a nontrivial kernel [27, chapter 10]. Tables I and II provide the timing and error results for CG on different formulations of the ranking problem. In this table, the edge and triangle densities are with respect to the number of edges and all possible triangles in a complete graph. The reported errors are measured relative to the known exact solution except in cases identified by an asterisk (*) in $\|h\|$ column where the corresponding error is an absolute one. These are cases for which the homology of the simplicial 2-complex induced from the graph is trivial leading to a zero harmonic component. The relative error column reports error in the norm of the gradient part ($\|\partial_1^T \alpha\|$), norm of the curl part ($\|\partial_2 \beta\|$), and the norm of the harmonic part ($\|h\|$). The timing labeled α shows the iterations and time required for the first least squares problem, and the one labeled β shows these for the second least squares problem.

C. Algebraic multigrid methods

Multigrid methods work by creating a hierarchy of linear systems of decreasing sizes from the given problem. At any level in the hierarchy, a larger system is called a *fine grid* and a smaller system is called a *coarse grid*. The solutions of the different systems are related via prolongation (coarse to fine) and restriction (fine to coarse) maps. In geometric multigrid, coarser levels correspond to a coarser mesh. In *algebraic* multigrid, coarsening is carried out using only the matrix and an associated adjacency graph. Coarsening is performed by aggregating those vertices of this graph which have a strong connection. The strength of connection is defined in various ways for different schemes [28]. In smoothed aggregation, a vertex can belong fractionally to several aggregates [29]. In Lloyd aggregation, the number of connections between vertices and the centers of their aggregates is minimized [30].

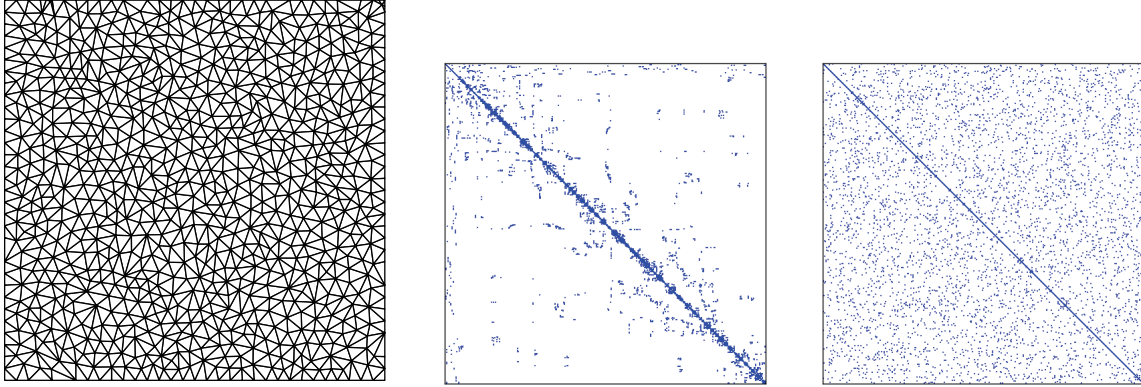


Figure 3: For the mesh shown on the left, the middle plot shows the nonzeros in Δ_0 . The rightmost plot shows the nonzeros in Δ_0 for an Erdős-Rényi graph constructed using the same number of vertices and edges as are in the mesh. The Δ_0 on the mesh has locality whereas the one on the graph does not.

We used algebraic multigrid with smoothed aggregation and Lloyd aggregation for the ranking problem on the Erdős-Rényi and Barabási-Albert random graphs. The parameters for the various graphs are identical to ones used in the experiments with Krylov solvers and the results are also given in Tables I and II. In the Algorithm/Formulation column, the notation AMG (SA) and AMG (LA) are used to indicate the smoothed and Lloyd aggregation schemes. As before, in these tables, the cases where the homology of the simplicial 2-complex is trivial are marked by an asterisk (*). The cases where PyAMG failed in the setup phase are marked by a dagger symbol (†).

D. Discussion and comparisons

From the results in Tables I and II, it is clear that conjugate gradient is almost always a good choice for the two least squares problems. Algebraic multigrid performs optimally for certain types of elliptic partial differential equations on meshes. So a naive hope would be that it would do well with Laplacians on graphs. Our experiments demonstrate that although algebraic multigrid can sometimes solve the linear systems, it often performs quite poorly in terms of time as compared to Krylov methods. This is true even if the setup time required by algebraic multigrid is excluded. (The setup is the process of forming the coarser levels before solving can begin.) If the setup time is also included, the performance becomes much worse compared to Krylov methods. For example, in the cases corresponding to the last two rows of Table I, algebraic multigrid could not even be used, whereas Krylov methods performed reasonably well. In the same table, for the β problem in rows 4 and 5, algebraic multigrid took between 2 and over 400 seconds while Krylov methods took only a fraction of a second. A similar behavior is seen in the case of Barabási-Albert graphs as well in Table II. We suspect that algebraic multigrid performs remarkably poorly since there is no apparent (or

very little) structure or locality for the least squares matrices arise from graphs. This is very different from what one would observe in the case of Laplacians on meshes. Fig. 3 provides an instance of this contrast.

In the last decade or so there has been extensive work on solvers for diagonally dominant systems [31]–[33]. The linear system corresponding to (4) is diagonally dominant for any graph. However the second least squares problem in (5) will not in general be a diagonally dominant except when the graph is on a surface.

V. CONCLUSIONS AND FUTURE WORK

In this paper, we have presented the first detailed numerical studies of two least squares problems on graphs. We show that algebraic multigrid which has been successful in solving linear systems arising from certain elliptic partial differential equations performs poorly for least squares problems on graphs. This may appear surprising since the two problems are versions of Poisson’s equations, one involving the scalar Laplacian and the other involving the Laplace-deRham operator on 2-cochains. However, as alluded to earlier and as can be seen from Fig. 3, the linear systems in graph problems can suffer from lack of locality and structure which is usually apparent in partial differential equation problems on meshes.

For the first least squares problem, our results indicate that using the conjugate gradient solver on the normal equations is always the best option, generating the lowest errors and the fastest times for each of the three types of graphs we considered. For the second least squares problem, conjugate gradient is again the better option except on some small sized graphs with fewer edges and triangles in them.

An interesting area for future research is the decomposition of these problems into smaller pieces which can be solved with minimal interaction. One then also needs to combine the solutions thus obtained into a global solution.

Table I: Comparison of solution of normalized form of least squares on Erdős-Rényi graphs using conjugate gradients and algebraic multigrid. A † in the table denotes that the method failed to produce a solution. A * denotes that the computed error is an absolute one and corresponds to cases where the harmonic is zero.

N_0	N_1	N_2	Edge Density	Triangle Density	Algorithm / Formulation	Relative Error			α iter.	α sec.	β iter.	β sec.
						$\ \partial_1^T \alpha\ $	$\ \partial_2 \beta\ $	$\ h\ $				
100	380	52	7.68e-02	3.22e-04	CG	1.1e-08	6.3e-09	2.0e-07	27	0.0041	17	0.0027
					AMG (SA)	5.0e-09	3.8e-09	9.5e-08	2	0.0577	2	0.0030
					AMG (LA)	5.0e-09	3.8e-09	9.5e-08	2	0.0078	2	0.0023
100	494	144	9.98e-02	8.91e-04	CG	1.1e-08	1.0e-08	2.9e-07	21	0.0037	30	0.0053
					AMG (SA)	2.8e-09	3.8e-09	1.0e-07	2	0.0078	2	0.0137
					AMG (LA)	2.8e-09	3.8e-09	1.0e-07	2	0.0082	2	0.0131
100	1212	2359	2.45e-01	1.46e-02	CG	3.9e-09	1.6e-08	3.5e-05	14	0.0023	60	0.0191
					AMG (SA)	1.2e-10	4.8e-08	1.1e-04	2	0.0079	32	0.6259
					AMG (LA)	1.2e-10	3.4e-08	7.5e-05	2	0.0079	30	0.1426
100	2530	21494	5.11e-01	1.33e-01	CG	2.5e-09	9.2e-09	3.0e-06*	10	0.0017	17	0.1033
					AMG (SA)	6.5e-10	6.9e-09	2.3e-06*	2	0.0142	27	2.102
					AMG (LA)	6.5e-10	4.8e-09	1.6e-06*	2	0.0184	19	15.89
100	3706	67865	7.49e-01	4.20e-01	CG	2.1e-09	6.6e-09	5.6e-06*	8	0.0015	11	0.4759
					AMG (SA)	2.8e-10	7.7e-09	6.6e-06*	2	0.0689	39	16.74
					AMG (LA)	2.8e-10	7.3e-09	6.2e-06*	2	0.0356	14	408.5
500	1290	21	1.03e-02	1.01e-06	CG	1.4e-08	1.9e-09	3.6e-07	43	0.0072	3	0.0007
					AMG (SA)	3.8e-09	1.9e-09	9.8e-08	2	0.7113	2	0.0360
					AMG (LA)	3.8e-09	1.9e-09	9.8e-08	2	0.3080	2	0.0022
500	12394	20315	9.94e-02	9.81e-04	CG	3.3e-09	2.7e-08	8.9e-05	13	0.0030	105	0.2155
					AMG (SA)	7.2e-11	7.7e-08	2.5e-04	2	0.3305	74	2.711
					AMG (LA)	7.2e-11	7.4e-08	2.5e-04	2	0.3293	71	8.358
500	24788	162986	1.99e-01	7.87e-03	CG	1.9e-09	1.3e-08	1.0e-05*	11	0.0032	25	1.447
					AMG (SA)	†	†	†	†	†	†	†
					AMG (LA)	†	†	†	†	†	†	†
1000	49690	163767	9.95e-02	9.86e-04	CG	1.9e-09	1.4e-08	2.5e-03	11	0.0049	52	1.916
					AMG (SA)	†	†	†	†	†	†	†
					AMG (LA)	†	†	†	†	†	†	†

It is not at all obvious how this can be done. For partial differential equations, smaller subproblems are sometimes obtained by domain decomposition methods. Analogously, appropriate graph partitioning methods and their role in decomposing these graph problems may be worth studying. These might also be relevant in creating new aggregation schemes for algebraic multigrid. For the least squares problem in (4) use of other solvers for diagonally dominant systems [31]–[33] would also be interesting to study further.

ACKNOWLEDGMENT

The work of ANH and KK was supported in part by NSF Grant No. DMS-0645604. We thank Rich Lehoucq and Han Wang for useful discussions.

REFERENCES

- [1] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, *LAPACK Users' Guide*, 3rd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 1999.
- [2] R. Barrett, M. Berry, T. F. Chan, and et al., *Templates for the solution of linear systems: building blocks for iterative methods*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 1994. [Online]. Available: <http://dx.doi.org/10.1137/1.9781611971538>
- [3] R. B. Lehoucq, D. C. Sorensen, and C. Yang, *ARPACK users' guide*, ser. Software, Environments, and Tools. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 1998, vol. 6, solution of large-scale eigenvalue problems with implicitly restarted Arnoldi methods. [Online]. Available: <http://dx.doi.org/10.1137/1.9780898719628>
- [4] Z. Bai, J. Demmel, J. Dongarra, A. Ruhe, and H. van der Vorst, Eds., *Templates for the solution of algebraic eigenvalue problems*, ser. Software, Environments, and Tools. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2000, vol. 11, a practical guide. [Online]. Available: <http://dx.doi.org/10.1137/1.9780898719581>
- [5] A. Buluç and J. R. Gilbert, “The combinatorial blas: design, implementation, and applications,” *International Journal of High Performance Computing Applications*, vol. 25, no. 4, pp. 496–509, 2011. [Online]. Available: <http://hpc.sagepub.com/content/25/4/496.abstract>
- [6] A. Lugowski, D. Alber, A. Buluç, J. R. Gilbert, S. Reinhardt, Y. Teng, and A. Waranis, “A flexible open-source toolbox

Table II: Comparison of solution of normalized form of least squares on Barabási-Albert graphs using conjugate gradients and algebraic multigrid. A † in the table denotes that the method failed to produce a solution. A * denotes that the computed error is an absolute one and corresponds to cases where the harmonic is zero.

N_0	N_1	N_2	Edge Density	Triangle Density	Algorithm / Formulation	Relative Error			iter.	α		iter.	β	
						$\ \partial_1^T \alpha\ $	$\ \partial_2 \beta\ $	$\ h\ $		sec.			sec.	
100	475	301	9.60e-02	1.86e-03	CG	1.1e-08	2.4e-08	1.3e-06	25	0.0038		79	0.0127	
					AMG (SA)	3.2e-09	4.3e-09	2.4e-07	2	0.0009		2	0.0038	
					AMG (LA)	3.2e-09	4.3e-09	2.4e-07	2	0.0011		2	0.0037	
100	900	1701	1.82e-01	1.05e-02	CG	1.1e-08	2.6e-08	1.0e-05	21	0.0033		96	0.0281	
					AMG (SA)	6.7e-10	4.7e-08	1.8e-05	2	0.0016		50	0.0995	
					AMG (LA)	6.7e-10	5.6e-08	2.2e-05	2	0.0009		44	0.1050	
100	1600	8105	3.23e-01	5.01e-02	CG	6.7e-09	1.3e-08	2.2e-06*	20	0.0032		50	0.0967	
					AMG (SA)	2.0e-09	1.2e-08	1.9e-06*	2	0.0009		14	0.3561	
					AMG (LA)	2.0e-09	5.9e-09	9.8e-07*	2	0.0010		12	1.545	
100	2400	24497	4.85e-01	1.51e-01	CG	5.2e-09	1.2e-08	4.3e-06*	18	0.0030		27	0.2435	
					AMG (SA)	1.1e-09	7.3e-09	2.6e-06*	2	0.0017		24	2.632	
					AMG (LA)	1.1e-09	7.6e-09	2.7e-06*	2	0.0013		13	22.41	
500	4900	4740	3.93e-02	2.29e-04	CG	1.0e-08	3.4e-08	9.1e-06	32	0.0059		165	0.0956	
					AMG (SA)	2.3e-09	7.5e-08	2.0e-05	2	0.0169		89	0.4003	
					AMG (LA)	2.3e-09	8.9e-08	2.4e-05	2	0.0137		95	0.5046	
500	9600	25016	7.70e-02	1.21e-03	CG	8.4e-09	3.0e-08	5.8e-05	27	0.0056		215	1.043	
					AMG (SA)	4.9e-10	7.3e-07	1.4e-03	2	0.0142		101 [†]	4.627	
					AMG (LA)	4.9e-10	4.0e-07	7.8e-04	2	0.0142		101 [†]	29.74	
500	18400	133933	1.47e-01	6.47e-03	CG	4.5e-09	2.2e-08	2.0e-03	23	0.0058		111	7.204	
					AMG (SA)	†	†	†	†	†		†	†	
					AMG (LA)	†	†	†	†	†		†	†	
1000	9900	6264	1.98e-02	3.77e-05	CG	7.4e-09	4.4e-08	1.1e-05	41	0.0091		211	0.1410	
					AMG (SA)	5.3e-09	5.6e-06	1.4e-03	5	0.0477		101 [†]	0.5723	
					AMG (LA)	3.5e-09	9.1e-06	2.3e-03	5	0.0682		101 [†]	1.22	
1000	19600	37365	3.92e-02	2.25e-04	CG	9.0e-09	4.1e-08	5.8e-05	33	0.0088		335	2.521	
					AMG (SA)	7.5e-09	1.1e-05	1.5e-02	4	0.0393		101 [†]	10.61	
					AMG (LA)	5.6e-09	8.6e-06	1.2e-02	4	0.0823		101 [†]	63.01	
1000	38400	202731	7.69e-02	1.22e-03	CG	5.7e-09	3.0e-08	5.2e-04	27	0.0096		211	20.65	
					AMG (SA)	†	†	†	†	†		†	†	
					AMG (LA)	†	†	†	†	†		†	†	

for scalable complex graph analysis,” in *Proceedings of the 2012 SIAM International Conference on Data Mining*, 2012, pp. 930–941. [Online]. Available: <http://epubs.siam.org/doi/abs/10.1137/1.9781611972825.80>

- [7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*, 3rd ed. MIT Press, Cambridge, MA, 2009.
- [8] X. Jiang, L.-H. Lim, Y. Yao, and Y. Ye, “Statistical ranking and combinatorial hodge theory,” *Mathematical Programming*, vol. 127, pp. 203–244, 2011. [Online]. Available: <http://dx.doi.org/10.1007/s10107-010-0419-x>
- [9] A. Giridhar and P. Kumar, “Distributed clock synchronization over wireless networks: Algorithms and analysis,” in *Decision and Control, 2006 45th IEEE Conference on*, dec. 2006, pp. 4915–4920.
- [10] R. Solis, V. Borkar, and P. Kumar, “A new distributed time synchronization protocol for multihop wireless networks,” in *Decision and Control, 2006 45th IEEE Conference on*, dec. 2006, pp. 2734–2739.
- [11] D. N. Arnold, R. S. Falk, and R. Winther, “Finite element exterior calculus: from Hodge theory to numerical stability,” *Bull. Amer. Math. Soc. (N.S.)*, vol. 47, no. 2, pp. 281–354, 2010.
- [12] P. S. Vassilevski and L. T. Zikatanov, “Commuting projections on graphs,” *Numer. Linear Algebra Appl.*, vol. 21, no. 3, pp. 297–315, 2014. [Online]. Available: <http://dx.doi.org/10.1002/nla.1872>
- [13] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: Bringing order to the web,” Stanford InfoLab, Technical Report 1999-66, November 1999, previous number = SIDL-WP-1999-0120. [Online]. Available: <http://ilpubs.stanford.edu:8090/422/>
- [14] S. Negahban, S. Oh, and D. Shah, “Iterative ranking from pair-wise comparisons,” in *Advances in Neural Information Processing Systems 25*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds. Curran Associates, Inc., 2012, pp. 2474–2482. [Online]. Available: <http://papers.nips.cc/paper/4701-iterative-ranking-from-pair-wise-comparisons.pdf>

- [15] The graph 500 list. Accessed January 2015. [Online]. Available: <http://www.graph500.org/>
- [16] R. J. Leake, “A method for ranking teams: With an application to college football,” in *Management Science in Sports*, ser. TIMS Studies in the Management Sciences, R. E. Machol and S. P. Ladany, Eds. North-Holland Publishing Company, 1976, vol. 4, pp. 27–46.
- [17] F. R. K. Chung, *Spectral Graph Theory*, ser. CBMS Regional Conference Series in Mathematics. Published for the Conference Board of the Mathematical Sciences, Washington, DC, 1997, vol. 92.
- [18] D. A. Spielman, “Algorithms, graph theory, and linear equations in Laplacian matrices,” in *Proceedings of the International Congress of Mathematicians*, 2010, pp. 2698–2722.
- [19] J. R. Munkres, *Elements of Algebraic Topology*. Menlo Park: Addison–Wesley Publishing Company, 1984.
- [20] R. Abraham, J. E. Marsden, and T. Ratiu, *Manifolds, Tensor Analysis, and Applications*, 2nd ed. New York: Springer–Verlag, 1988.
- [21] M. T. Heath, *Scientific Computing: An Introductory Survey*, 2nd ed. McGraw–Hill, 2002.
- [22] P. Erdős and A. Rényi, “On the evolution of random graphs,” *Magyar Tud. Akad. Mat. Kutató Int. Közl. (Publ. Math. Inst. Hung. Acad. Sci.)*, vol. 5, pp. 17–61, 1960.
- [23] A. Barabási and R. Albert, “Emergence of scaling in random networks,” *Science*, vol. 286, no. 5439, pp. 509–512, 1999.
- [24] E. Jones, T. Oliphant, and P. Peterson, “SciPy: Open source scientific tools for Python,” 2001. [Online]. Available: <http://www.scipy.org/>
- [25] W. N. Bell, L. N. Olson, and J. B. Schroder, “PyAMG: Algebraic multigrid solvers in Python v2.0,” 2011, release 2.0. [Online]. Available: <http://www.pyamg.org>
- [26] N. Bell and A. N. Hirani, “PyDEC: Software and algorithms for Discretization of Exterior Calculus,” *ACM Transactions on Mathematical Software*, vol. 39, no. 1, pp. 3:1 – 3:41, 2012.
- [27] H. A. van der Vorst, *Iterative Krylov Methods for Large Linear Systems*, ser. Cambridge Monographs on Applied and Computational Mathematics. Cambridge: Cambridge University Press, 2009, vol. 13, reprint of the 2003 original.
- [28] K. Stüben, “A review of algebraic multigrid,” *Journal of Computational and Applied Mathematics*, vol. 128, pp. 281–309, 2001.
- [29] P. Vaněk, J. Mandel, and M. Brezina, “Algebraic Multigrid by Smoothed Aggregation for Second and Fourth Order Elliptic Problems,” *Computing*, vol. 56, no. 3, pp. 179–196, 1996.
- [30] W. N. Bell, “Algebraic multigrid for discrete differential forms,” Ph.D. dissertation, University of Illinois at Urbana-Champaign, Urbana, Illinois, 2008. [Online]. Available: <http://graphics.cs.uiuc.edu/~wnbell/publications/2008-08-Thesis/Bell2008.pdf>
- [31] D. A. Spielman and S.-H. Teng, “Nearly-linear time algorithms for graph partitioning, graph sparsification, and solving linear systems,” in *Proceedings of the 36th Annual ACM Symposium on Theory of Computing*. New York: ACM, 2004, pp. 81–90. [Online]. Available: <http://dx.doi.org/10.1145/1007352.1007372>
- [32] I. Koutis, G. Miller, and R. Peng, “Approaching optimality for solving SDD systems,” in *Proceedings of Foundations of Computer Science*, 2010, pp. 235–244.
- [33] J. A. Kelner, L. Orecchia, A. Sidford, and Z. A. Zhu, “A simple, combinatorial algorithm for solving SDD systems in nearly-linear time,” in *STOC’13—Proceedings of the 2013 ACM Symposium on Theory of Computing*. ACM, New York, 2013, pp. 911–920. [Online]. Available: <http://dx.doi.org/10.1145/2488608.2488724>